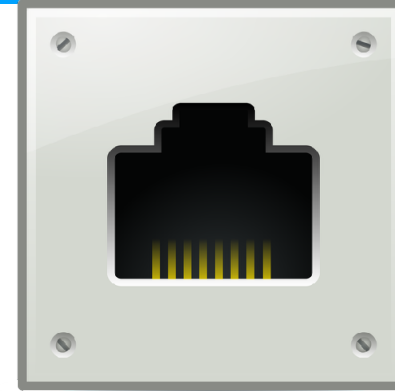


Esercitazioni Ing.Sw

Gian Enrico Conti

Mar 2026

Simple TCP Cl/Srv



Prova Finale - Ingegneria del Software

Ports

$\text{IP } x : \text{port } y \text{ — IP } w : \text{Port } z$

$x \neq w ?$

$y \neq z ?$

tutto chiaro?

Sample:

<https://docs.oracle.com/javase/tutorial/networking/sockets/>

In particular:

<https://docs.oracle.com/javase/tutorial/displayCode.html?code=https://docs.oracle.com/javase/tutorial/networking/sockets/examples/EchoServer.java>

Code:

<https://docs.oracle.com/javase/tutorial/displayCode.html?code=https://docs.oracle.com/javase/tutorial/networking/sockets/examples/EchoServer.java>

Classes:

java.net.Socket

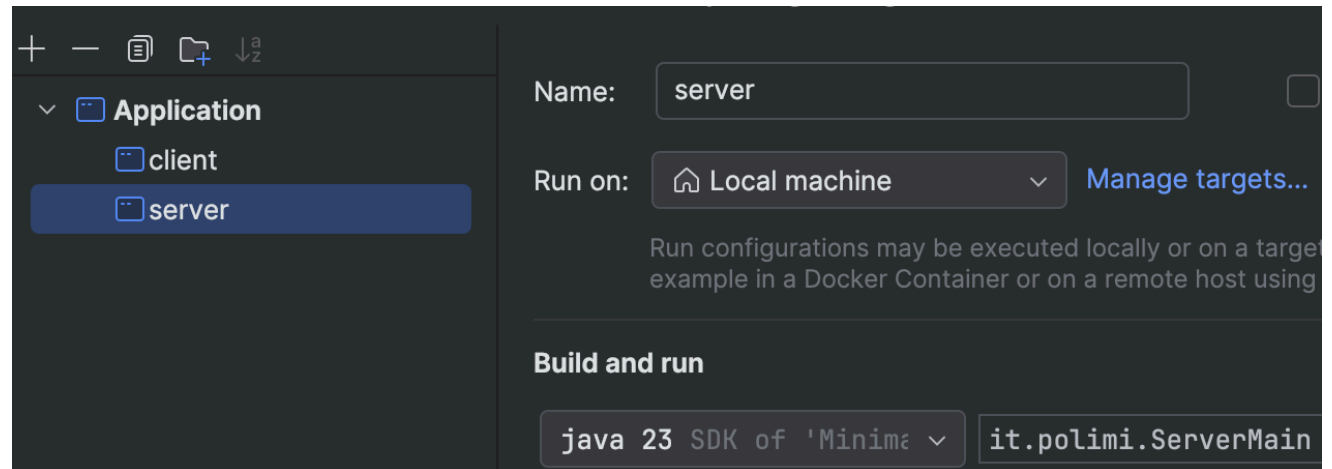
java.net.ServerSocket

Prima due step banali:

creare un project con 2 main:

- ClientMain
- ServerMain

e i relativi 2 setup
di test (intelliJ):



For Your convenience..

<https://github.com/ingconti/MinimalTCPServerAndClient>

un run di test..

- 📌 master
- 🔗 01-keep-server-running
- 🔗 02-client-server_fixed
- 🔗 03-multithreaded
- 🔗 04-JSONCmds
- 🔗 05-parseJSONCmds
- 🔗 06-jsonparsing_FIX
- 🔗 07-process_multipleCmds

server Code

```
static int portNumber = 1234;  
  
public static void main(String[] args) {  
    System.out.println("Server started!");  
  
    ..  
}
```

Wait!

Try / Catch

- **UN solo** try/catch (catch all) ?
- **singoli** try/catch?
- se unico sembra piu facile ma alla fine
NON testate *decentemente* i vari casi...
- piu try/catch:
 - molto piu flessibile.. (anche se pare confuso..)
 - piu' debuggabile
 - piu' testabile
 - recover selettivo
 - diagnostica piu' accurata

se poi troppo confuso, SEPARATE i METODI!

Note

```
static int portNumber = 1234;
```

Bad practice.

Use:

```
String hostName = args[0];  
int portNumber = Integer.parseInt(args[1]);
```

Or better:

```
String hostName = Prefs.ReadHostFromJSON();  
int portNumber = Prefs.ReadPortFromJSON();
```

Or best:

```
...  
If (argc==2){  
    hostName = args[0];  
    portNumber = Integer.parseInt(args[1]);  
}else{  
    hostName = Prefs.ReadHostFromJSON();  
    portNumber = Prefs.ReadPortFromJSON();  
}
```

```
public class ServerMain {
    static int portNumber = 1234;

    public static void main(String[] args) {

        ServerSocket serverSocket = null;

        try {
            serverSocket = new ServerSocket(portNumber);
        } catch (IOException e) {
            e.printStackTrace();
        }

        System.out.println("Listening on port " + portNumber);

        Socket clientSocket = null;
        try {
            clientSocket = serverSocket.accept();
        } catch (IOException e) {
            e.printStackTrace();
        }

        BufferedReader in = null;
        try {
            in = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream()));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

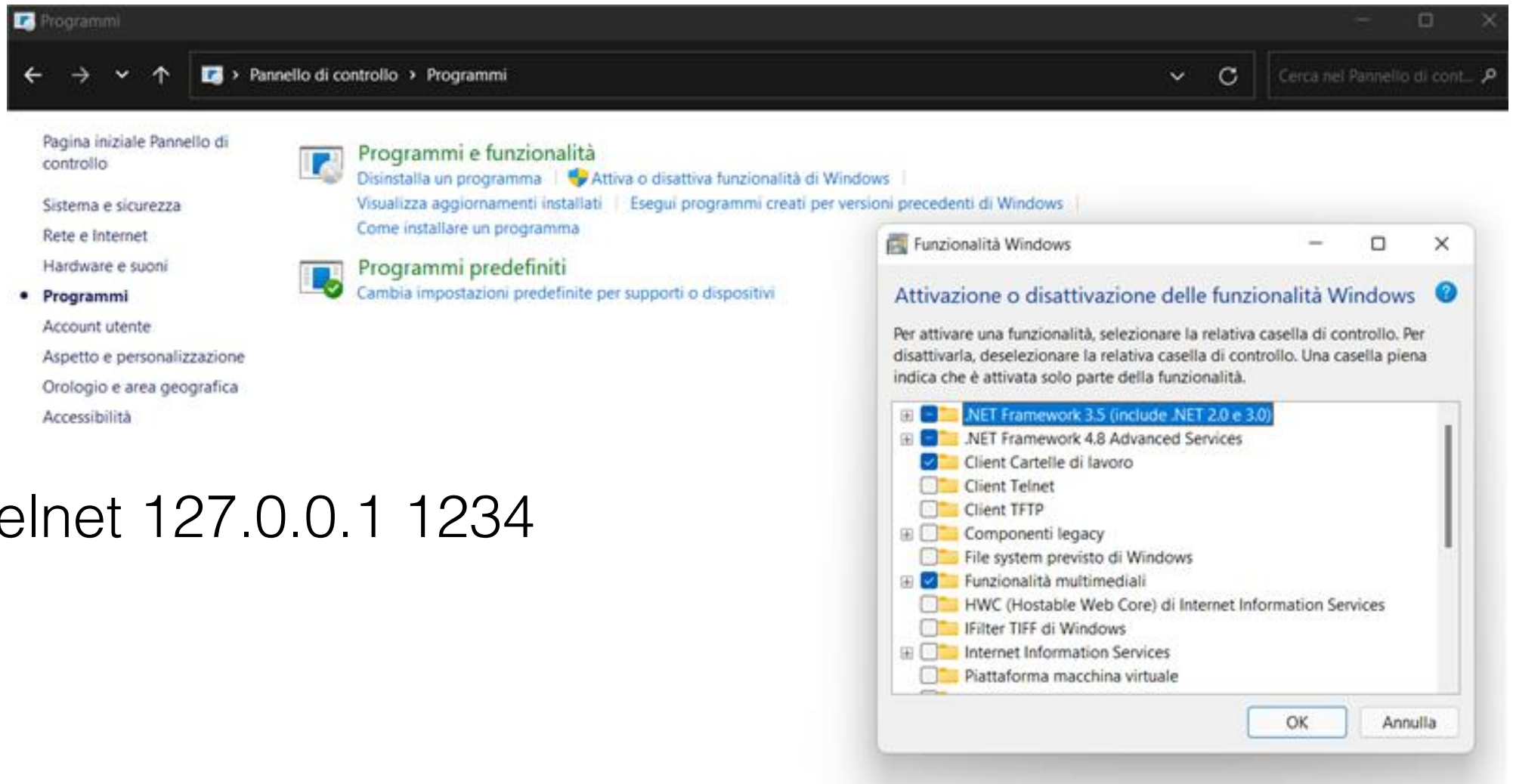
debuggin tips

- non scrivete 1000 righe e run
- il vs codice DEVE funzionare prima da cmd line

usate NC (o telnet..) !

```
/usr/bin/nc 127.0.0.1 1234
```

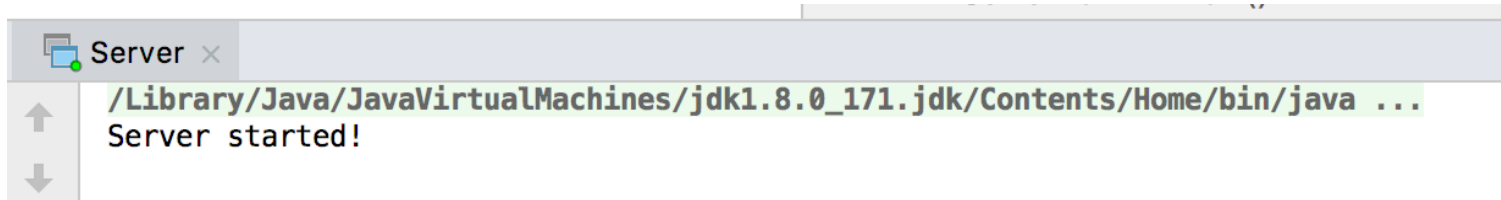
Debugging tips x Windows



telnet 127.0.0.1 1234

debuggin tips

run del server:



```
Server x  
/Library/Java/JavaVirtualMachines/jdk1.8.0_171.jdk/Contents/Home/bin/java ...  
Server started!
```

```
/usr/bin/nc 127.0.0.1 1234
```

```
/Library/Java/JavaVirtualMachines/jdk1.8.0_171.jdk/Contents/Home/bin/java ...  
Server started!  
accepting ..  
done.  
  
Process finished with exit code 0  
|
```

03-client-server_fixed

keep server running

```
try {
    in = new BufferedReader(
        new InputStreamReader(clientSocket.getInputStream()));
} catch (IOException e) {
    e.printStackTrace();
}

// waits for data and reads it in until connection dies
// readLine() blocks until the server receives a new line from client
String s = "";
try {
    while ((s = in.readLine()) != null) {
        System.out.println(s);
    }

} catch (IOException e) {
    e.printStackTrace();
}
```

(taken from: <https://introcs.cs.princeton.edu/java/84network/EchoServer.java.html>)

ANOTHER RUN..

keep server running: code

keep-server-running

```
public class ServerMain {
    static int portNumber = 1234;
    public static void main(String[] args) {

        System.out.println("Server Started!");

        ServerSocket serverSocket = null;
        try {
            serverSocket = new ServerSocket(portNumber);
        } catch (IOException e) {
            e.printStackTrace();
        }

        System.out.println("Listening on port " + portNumber);
        Socket clientSocket = null;
        try {
            clientSocket = serverSocket.accept();
        } catch (IOException e) {
            e.printStackTrace();
        }

        System.out.println("Accepted");

        BufferedReader in = null;
        try {
            in = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream()));
        } catch (IOException e) {
            e.printStackTrace();
        }

        // waits for data and reads it in until connection dies
        // readLine() blocks until the server receives a new line from client
        String s = "";
        try {
            while ((s = in.readLine()) != null) {
                System.out.println(s);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        // we should close..
    }
}
```



Client Code

```
public class ClientMain {
    public static void main(String[] args) {

        String hostName = "127.0.0.1";
        int portNumber = 1234;

        Socket echoSocket = null;
        try {
            echoSocket = new Socket(hostName, portNumber);
        }
        catch (IOException e) {
            System.err.println(e.toString() + " " + hostName);
            System.exit(1);
        }

        PrintWriter out = null;
        BufferedReader in = null;
        BufferedReader stdIn = null;
        try {
            out = new PrintWriter(echoSocket.getOutputStream(), true);
            in = new BufferedReader(new InputStreamReader(echoSocket.getInputStream()));
            stdIn = new BufferedReader(new InputStreamReader(System.in));
        } catch (IOException e) {
            System.err.println(e.toString() + " " + hostName);
            System.exit(1);
        }

        String userInput = "";
        while (true) {
            try {
                if (!(userInput = stdIn.readLine()) != null) break;
                out.println(userInput);
                System.out.println("echo: " + in.readLine());
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        } // while
    }
}
```

**Va fixata risposta
sul server..**

Server fix

🔗 client-server_fixed

```
public class ServerMain {
    static int portNumber = 1234;

    public static void main(String[] args) {

        System.out.println("Server Started!");
        ServerSocket serverSocket = null;
        try {
            serverSocket = new ServerSocket(portNumber);
        } catch (IOException e) {
            e.printStackTrace();
        }

        System.out.println("Listening on port " + portNumber);

        Socket clientSocket = null;
        try {
            clientSocket = serverSocket.accept();
        } catch (IOException e) {
            e.printStackTrace();
        }

        System.out.println("Accepted");

        BufferedReader in = null;
        PrintWriter out = null;
        try {
            in = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream()));
            out = new PrintWriter(clientSocket.getOutputStream(), true);
        } catch (IOException e) {
            e.printStackTrace();
        }

        // waits for data and reads it in until connection dies
        // readLine() blocks until the server receives a new line from client
        String s = "";
        try {
            while ((s = in.readLine()) != null) {
                System.out.println(s);
                out.println(s.toUpperCase());
            }
        } catch (IOException e) {
            e.printStackTrace();
        }

        // we should close..
    }
}
```

Fix

Fix

Blocking/Threading

l' approccio precedente gestisce UNO SOLO socket x volta...
(ed esce alla disconnessione..Quindi loop con accept..)

Alternative:

- il client “sgancia” ad ogni cmd (diventa *quasi http...*)
(**NON** e' il ns modello..)
- il server genera x ogni accept un thread
(piu corretto, non-blocking, MA poi **synchronized...**)

Make multi-threaded

Prima un po' di refactor..

Dalla riga:

```
BufferedReader in = null;
```

Spostiamo in altra f. ("Extract method") *clientLoop*

...

```
System.out.println("Accepted");
    clientLoop(clientSocket);
}

private static void clientLoop(Socket clientSocket) {
    BufferedReader in = null;
```

Make multi-threaded 2

Logica:

- X ogni accept, thread -> while
- il thread conterrà il loop

Classe: ClientHandler...

```
public class ClientHandler implements Runnable {  
    @Override  
    public void run() {  
        // qui clientLoop  
    }  
}
```

..

Passeremo socket nel costruttore quindi attributo

```
private Socket clientSocket;
```

Quindi..

Make multi-threaded: Code

🔗 03-multithreaded

```
public class ClientHandler implements Runnable {

    private Socket clientSocket;

    public ClientHandler(Socket clientSocket) {
        this.clientSocket = clientSocket;
    }

    @Override
    public void run() {
        clientLoop();
    }

    private void clientLoop() {
        BufferedReader in = null;
        PrintWriter out = null;
        try {
            in = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream()));
            out = new PrintWriter(clientSocket.getOutputStream(), true);
        } catch (IOException e) {
            e.printStackTrace();
        }

        // waits for data and reads it in until connection dies
        // readLine() blocks until the server receives
        // a new line from client
        String s = "";
        try {
            while ((s = in.readLine()) != null) {
                System.out.println(s);
                out.println(s.toUpperCase());
            }
        } catch (IOException e) {
            e.printStackTrace();
        }

        // we should close..
    }
}
```

```
public class ServerMain {
    static int portNumber = 1234;

    public static void main(String[] args) {

        System.out.println("Server Started!");
        ServerSocket serverSocket = null;
        try {
            serverSocket =
                new ServerSocket(portNumber);
        } catch (IOException e) {
            e.printStackTrace();
        }

        System.out.println(
            "Listening on port " + portNumber);
        // loop forever accepting..
        while (true) {
            Socket clientSocket = null;
            try {
                clientSocket = serverSocket.accept();
            } catch (IOException e) {
                e.printStackTrace();
            }

            System.out.println("Accepted");
            ClientHandler clientHandler =
                new ClientHandler(clientSocket);
            Thread t = new Thread(clientHandler);
            t.start();
        }
    }
}
```

Serializzazione: How To

- Serializzazione JAVA

 - Pro ..

 - Contro...

- Serializzazione "language Agnostic"

 - Pro ..

 - Contro...



Serializzazione: back on Code

JSON file: (will be added to Resources)

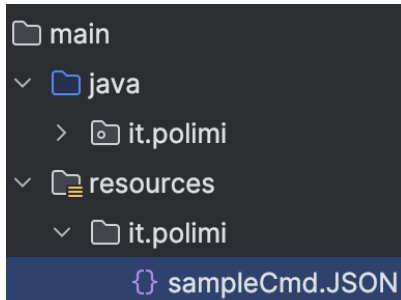
Supponiamo di usare JSON:

```
{  
  "command" : "moveTo",  
  "coords": {  
    "x": 1,  
    "y": 0  
  }  
}
```

E sul server parso il JSON....


```
public class Cmd {
    Coords coords;
    String command;

    @Override
    public String toString() {
        return "Cmd{" +
            "coords=" + coords +
            ", command='" + command
+ '\'' +
            "'}";
    }
}
```



POM:

```
<!-- jGson-->
<dependency>
    <groupId>com.google.code.gson</
groupId>
    <artifactId>gson</artifactId>
    <version>2.9.0</version>
</dependency>
```

```
public class Coords {
    long x;
    long y;

    @Override
    public String toString() {
        return "Coords{" +
            "x=" + x +
            ", y=" + y +
            "'}";
    }
}
```

```
class CmdTest {
    @Test
    void testJSON() {
        String input = "{\n" +
            "    \"command\" : \"moveTo\",\n" +
            "    \"coords\" : {\n" +
            "        \"x\" : 1,\n" +
            "        \"y\" : 0\n" +
            "    }\n" +
            "}";

        //System.out.println(input);
        Gson gson = new GsonBuilder().create();
        Cmd cmd = gson.fromJson(input, Cmd.class);

        assertTrue(cmd.command.equals("moveTo"));
        assertTrue(cmd.coords.x == 1);
    }
}
```

Serializzazione: RUN!

Run server ..

Da cmd LINE:

```
cat cmd.json | /usr/bin/nc 127.0.0.1 1234
```

Sul server appare: *(rispondiamo con maiuscolo al client. . out.println(s.toUpperCase());)*

```
{  
  "COMMAND" : "MOVETO",  
  "COORDS" : {  
    "X" : 1,  
    "Y" : 0  
  }  
}
```

parso il JSON, ma prima un modello che contenga i DATI..

Serializzazione: Model

modello che contenga i DATI..

```
public class Model {
    List<Coords> coords = new ArrayList<>();
    Gson gson = new GsonBuilder().create();

    // from tests..
    void addCoordsFromString(String jsonStr){
        Cmd cmd = gson.fromJson(jsonStr, Cmd.class);
        if (cmd.command.equals("moveTo")) {
            this.coords.add(cmd.coords);
        }
    }

    public String toString() {
        return "Model{" +
            "coords=" + coords +
            '}';
    }
}
```

PRIMA di metterlo nel codice un test....
(Molto simile al precedente..)

```
class ModelTest {

    @Test
    void addCoordsFromString() {
        String input = "{\n" +
            "  \"command\" : \"moveTo\",\n" +
            "  \"coords\": {\n" +
            "    \"x\": 1,\n" +
            "    \"y\": 0\n" +
            "  }\n" +
            "}";

        Gson gson = new GsonBuilder().create();
        Model model = new Model();
        model.addCoordsFromString(input);
        model.addCoordsFromString(input);

        List<Coords> coords = model.coords;
        assertTrue(coords.size() == 2);

        String wrong = "{\n" +
            "  \"command\" : \"WRONG\",\n" +
            "  \"coords\": {\n" +
            "    \"x\": 1,\n" +
            "    \"y\": 0\n" +
            "  }\n" +
            "}";

        coords = model.coords;
        assertTrue(coords.size() == 2);
    }
}
```

Nel main:

```
Model model = new Model();
System.out.println("Listening on port " + portNumber);
..
```

Va passato al clientHandler:

```
private Socket clientSocket;
private Model model;

public ClientHandler(Socket clientSocket, Model model) {
    this.clientSocket = clientSocket;
    this.model = model;
}
```

Nel main:

```
ClientHandler clientHandler = new ClientHandler(clientSocket, model);
```

Ed ancora nel Nel main:

```
try {
    while ((s = in.readLine()) != null) {
        System.out.println(s);
        this.model.addCoordsFromString(s);
        // no more... out.println(s.toUpperCase());
        out.println(model.toString());
    }
}
```

RUN..

Caused by: java.io.EOFException: End of input at line 1 column 2 path

.. il file ha gli a capo.. quindi al partner arriva "{".

FIX del JSON, one line da CMD LINE del Client:

```
{  "command" : "moveTo",  "coords": {    "x": 1,    "y": 0  } }
```

RUN..

Dopo 3 run da shell abbiamo:

```
Model{coords=[Coords{x=1, y=0}, Coords{x=1, y=0}, Coords{x=1, y=0}]}
```

Ed il client?

Serializzazione: Client e Cmds

Sarebbe sufficiente mandare 3 diverse stringhe di comando ad ogni invio, giusto per provare...

```
...
while (true) {
    try {
        if (!((userInput = stdIn.readLine()) != null)) break;
        // no more ..out.println(userInput);
        out.println(buildJSON(count++));
        System.out.println("echo: " + in.readLine());
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
} // while
}

static String buildJSON(int x){
    Cmd cmd = new Cmd();
    cmd.command = "moveTo";
    cmd.coords = new Coords(x, 100);    // added constructor.

    Gson gson = new GsonBuilder().create();
    String result = gson.toJson(cmd);
    return result;
}
```



Basta invio dal
terminale!

07-process_multipleCmds

RUN:

```
/Users/ingconti/Library/Java/JavaVirtualMachines/openjdk-23.0.2-1/Contents/Home/bin/java -javaagent:/Appl
```

```
echo: Model{coords=[Coords{x=1, y=100}]}
```

```
echo: Model{coords=[Coords{x=1, y=100}, Coords{x=2, y=100}]}
```

```
echo: Model{coords=[Coords{x=1, y=100}, Coords{x=2, y=100}, Coords{x=3, y=100}]}
```

```
echo: Model{coords=[Coords{x=1, y=100}, Coords{x=2, y=100}, Coords{x=3, y=100}, Coords{x=4, y=100}]}  
|
```

L'esempio appena visto contiene:

Server che risponde ai comandi

Server con un thread per ogni client (multi thread)

Parsing dei comandi

Mini Modello

Modello che "evolve"

Modello che "risponde"

Rete che rimanda esito al client

Client che aggiorna la vista (per ora terminale...)

Quindi....

(Vedi Towards Remote View2025_v3_reduced_Console_Cugola)

Client: e la grafica? Due app? Una App?

GRAFICA... Spoiler... ma giusto due idee..

Client: e la grafica? Due app? Una App?

GRAFICA:

- Basta rifare un client con "demo"
- si portano dentro le classi del client
- Occhio a runLater!

Una sola APP:

- si fa una app JAVA FX demo e si "condiziona" ai params da cmd line:

```
public class HelloApplication extends Application {
    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(HelloApplication.class.getResource("hello-view.fxml"));
        Scene scene = new Scene(fxmlLoader.load(), 320, 240);
        stage.setTitle("Hello!");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        if (args.length == 0) {
            launch(); // GUI...
            return;
        }
        int portNumber = Integer.parseInt(args[1]); // port listening to as server..

        System.out.println("Server Started!");
        ServerSocket serverSocket = null;
        try {
            serverSocket = new ServerSocket(portNumber);
        } catch (IOException e) {
            e.printStackTrace();
        }

        Model model = new Model();
        ...
    }
}
```